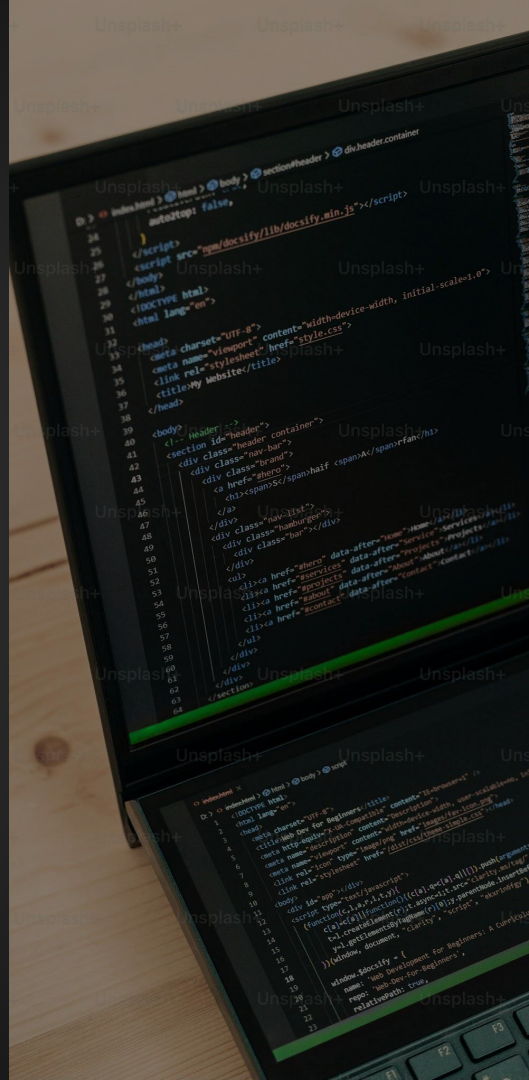


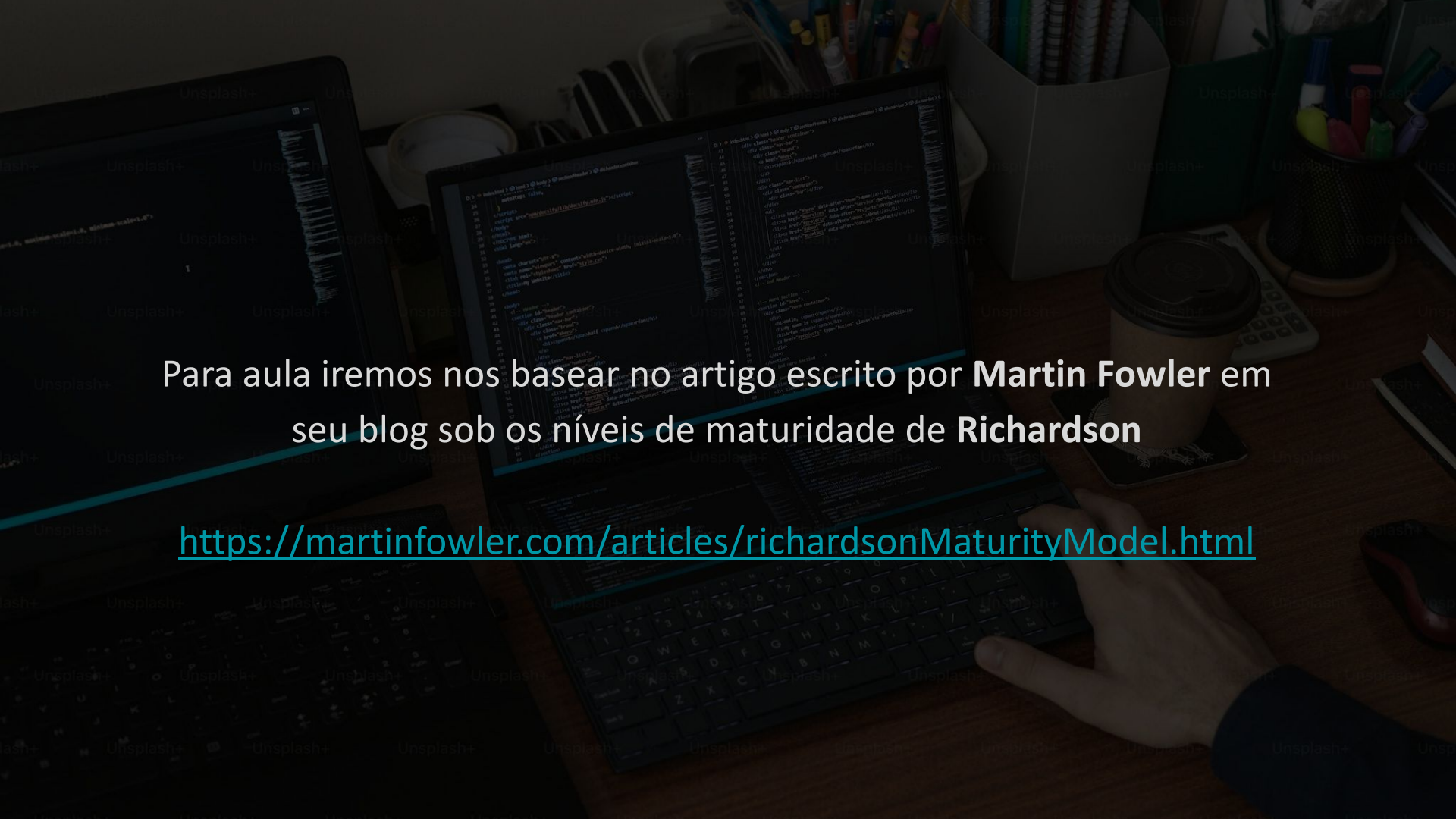
Desenvolvimento de Microserviços

Prof. Guilherme Alves

Richardson Maturity Model

Leonard Richardson fez uma análise em centenas de API's diferentes. E dividiu em quatro categorias para identificar o nível de maturidade da API com base no quanto eles são compatíveis com REST. Esse modelo é chamado de **Richardson Maturity Model**.



A photograph of a person's hand typing on a laptop keyboard. The laptop screen displays code in a dark-themed editor. The desk is cluttered with various items: a coffee cup, a calculator, a pen holder with pens and pencils, and a small container with more pens. The background is slightly blurred, showing more office supplies and a desk lamp.

Para aula iremos nos basear no artigo escrito por **Martin Fowler** em seu blog sob os níveis de maturidade de **Richardson**

<https://martinfowler.com/articles/richardsonMaturityModel.html>

Glory of REST



Level 3: Hypermedia Controls

Level 2: HTTP Verbs

Level 1: Resources

Level 0: The Swamp of POX

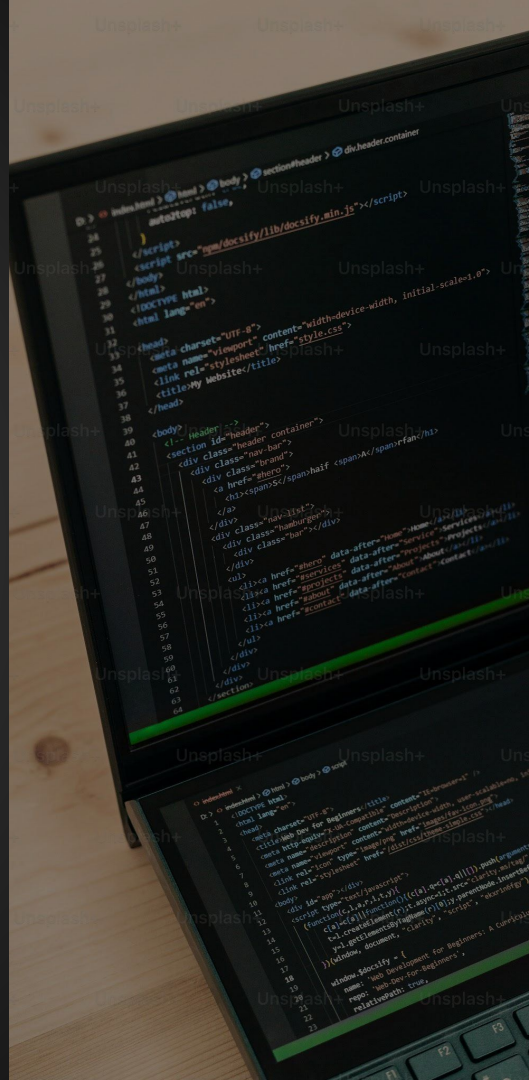


Nível 0

POX (*Plain Old XML*) ou então **O Anti-REST**

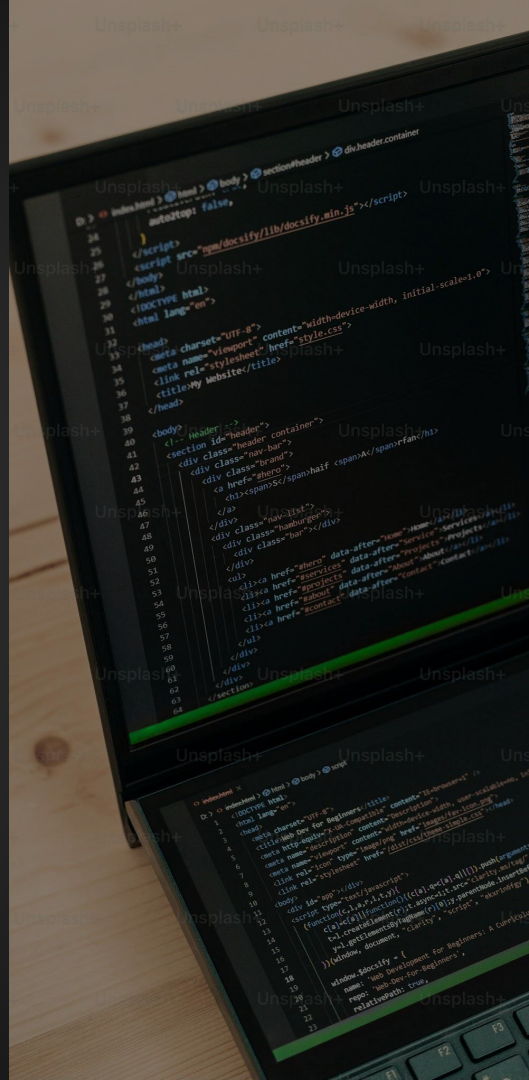
Esse é o nível mais baixo da maturidade de **APIs**, onde a **API** não aproveita os benefícios do **HTTP** corretamente.

Em vez de seguir a estrutura **REST**, ela funciona mais como um serviço remoto genérico, onde tudo é enviado para um único endpoint e os comandos são passados no corpo da requisição.



Nível 0

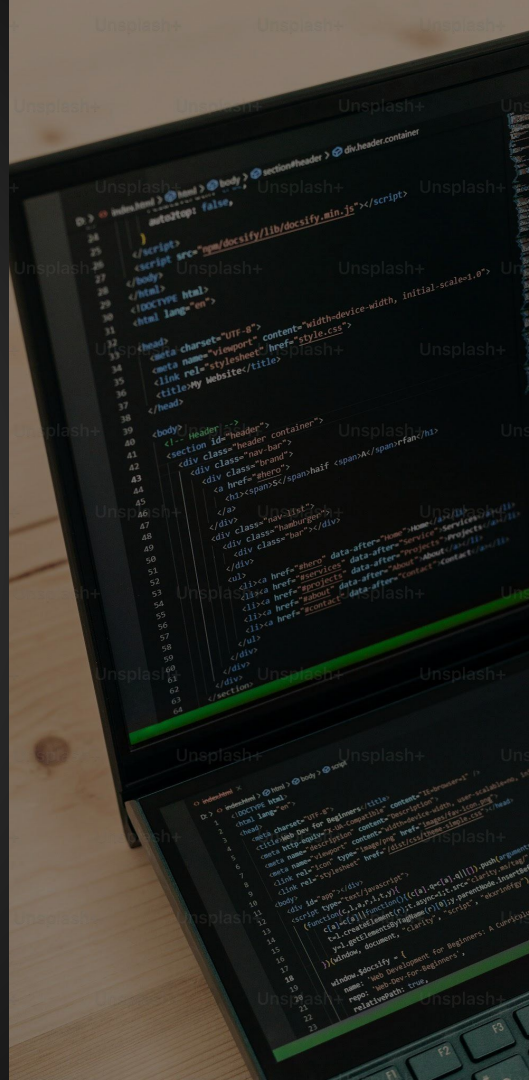
- Tudo é tratado como uma única ação genérica
- Métodos **HTTP** não são respeitados
- URLs não representam recursos
- Dificuldade em integrar e manter



Nível 1

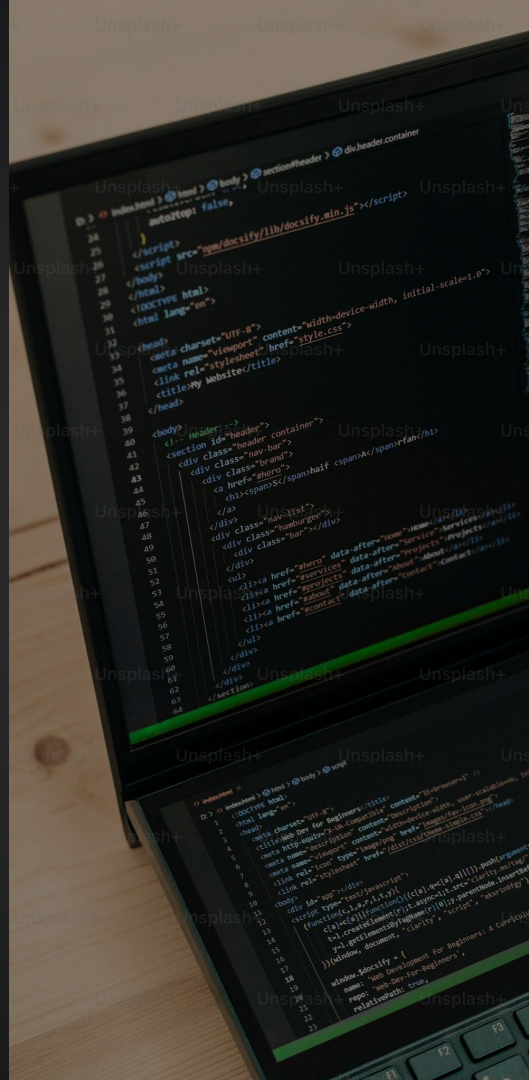
O nível um de maturidade já considera a utilização eficiente de **URIs**. Os recursos são mapeados, mas ainda não emprega o uso eficiente dos verbos. Geralmente utilizam apenas **GET** e **POST**.

Diria que a maioria das **API's** atualmente estão nesse nível de maturidade.



Nível 1

- **URLs** representam recursos
- Uso de um único endpoint por recurso
- Não há um uso adequado dos métodos **HTTP**

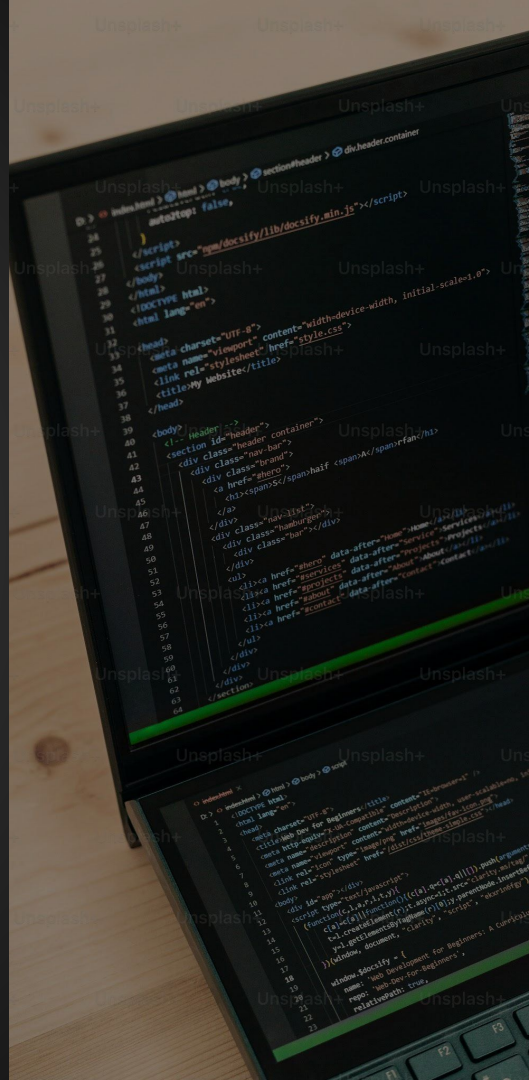


Nível 2

O nível dois de maturidade faz o uso eficiente de **URIs** e verbos **HTTP**.

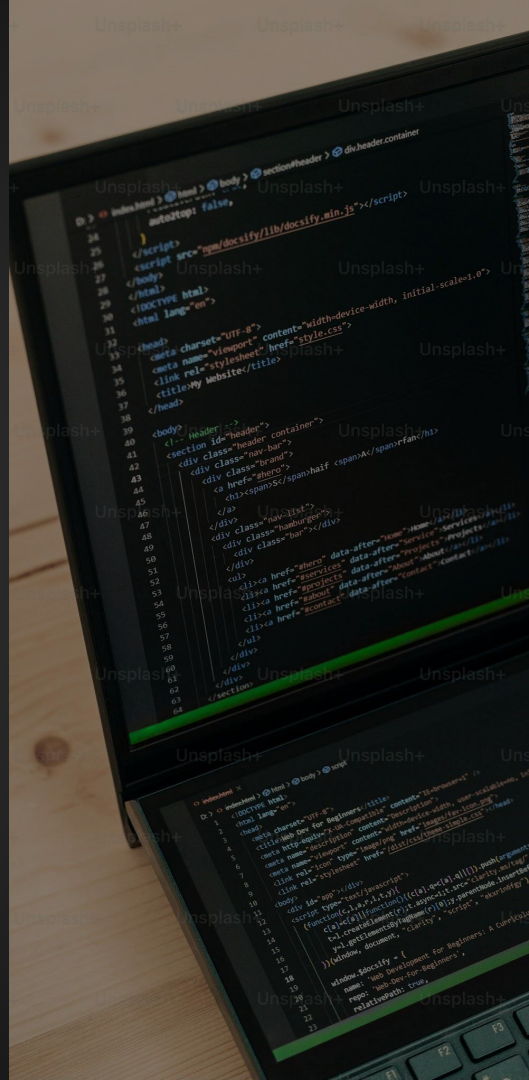
Nos níveis anteriores o protocolo **HTTP** estava sendo usado superficialmente. Nesse nível as **API's** começam a se aproximar do que o **RESTful** espera.

Neste nível, a **API** já entende que os diferentes métodos **HTTP** têm significados semânticos e deve usá-los corretamente de acordo com a operação que está sendo realizada.



Nível 2

- Uso correto dos métodos **HTTP**
- Códigos de status **HTTP**
- Respostas e ações semânticas
- A **URL** e o verbo **HTTP** devem trabalhar juntos

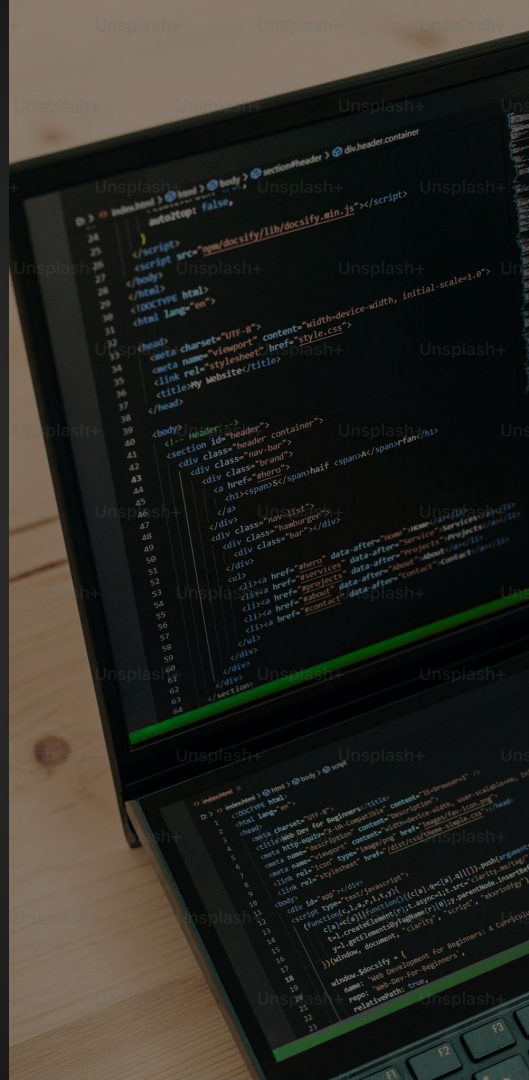


Nível 3

O nível três de maturidade faz o uso eficiente dos três fatores.

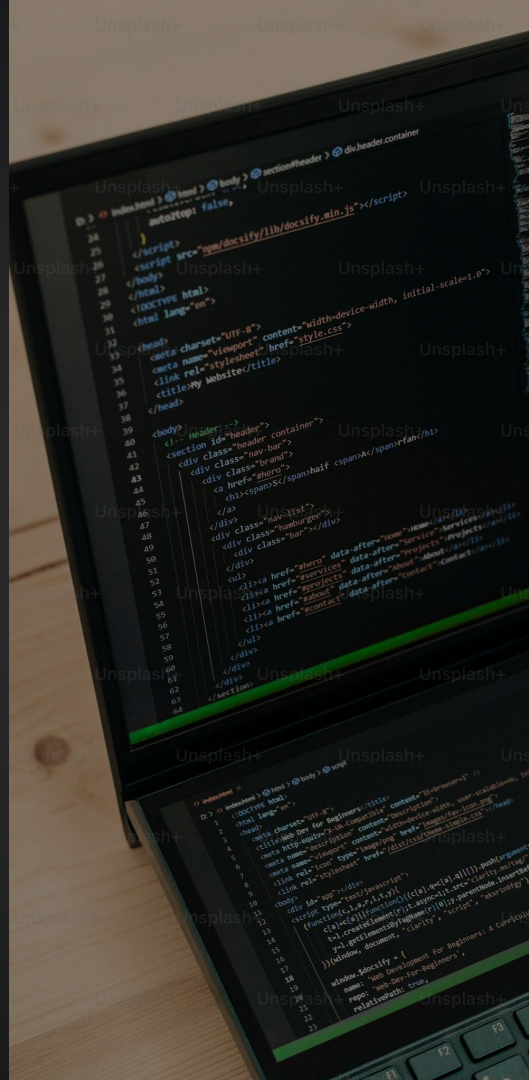
- URIs
- HTTP
- HATEOAS

O objetivo dos controles hipermídia é que eles nos digam o que podemos fazer a seguir e o **URI** do recurso que precisamos manipular para fazê-lo.



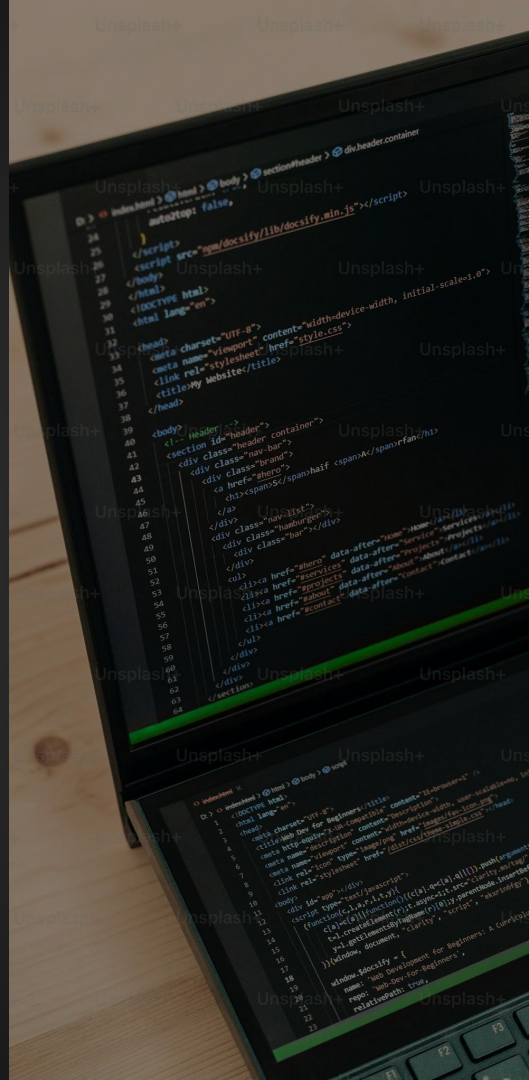
Nível 3

Esse nível representa uma das características mais avançadas e poderosas do estilo de arquitetura **RESTful**, onde a **API** começa a fornecer links dinâmicos dentro das respostas, permitindo que o cliente descubra a navegação através dos recursos da **API** sem precisar de documentação externa.



Nível 3

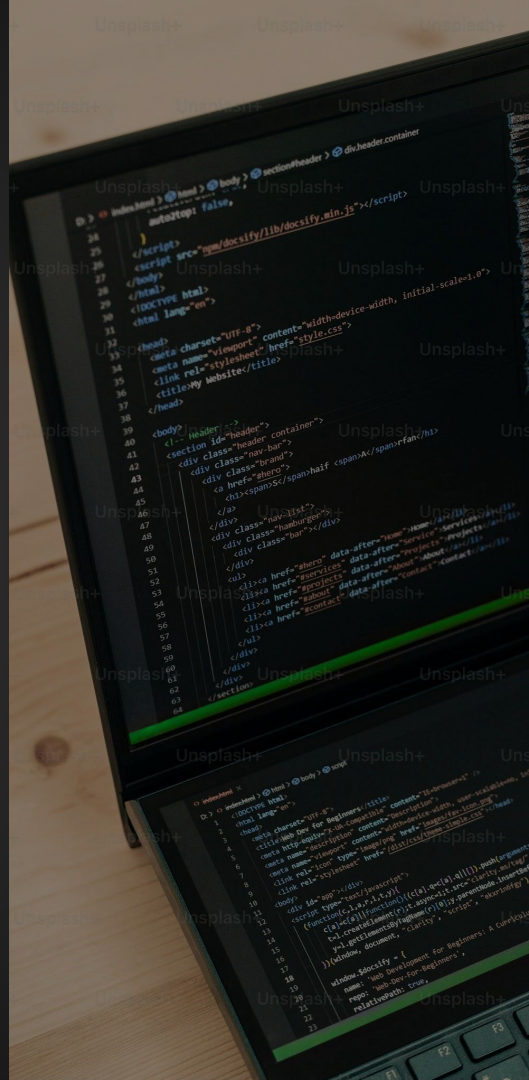
- Navegação dinâmica via hyperlinks
- Desacoplamento do cliente e da **API**
- Respostas enriquecidas com links
- Interação baseada em estado e contexto

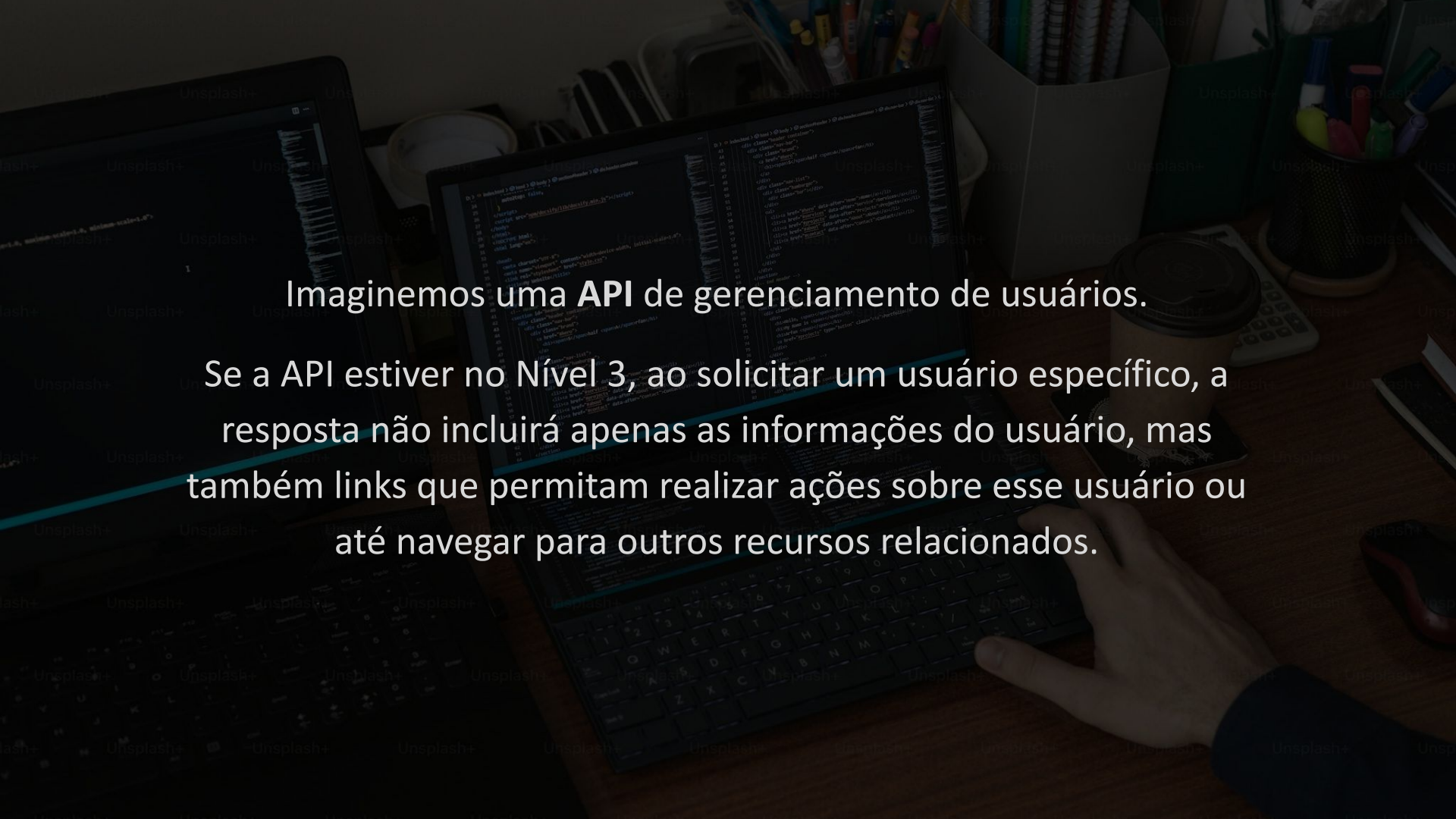


HATEOAS

HATEOAS (*Hypermedia as the Engine of Application State*), implica que, ao interagir com a **API**, o cliente não precisa saber de antemão os endpoints ou os recursos que poderá acessar.

Em vez disso, ele pode navegar pela aplicação seguindo links fornecidos na própria resposta das requisições.





Imaginemos uma **API** de gerenciamento de usuários.

Se a API estiver no Nível 3, ao solicitar um usuário específico, a resposta não incluirá apenas as informações do usuário, mas também links que permitam realizar ações sobre esse usuário ou até navegar para outros recursos relacionados.



```
1  {
2    "id": 123,
3    "nome": "João",
4    "email": "joao@email.com",
5    "_links": {
6      "self": {
7        "href": "/usuarios/123"
8      },
9      "update": {
10       "href": "/usuarios/123"
11     },
12     "delete": {
13       "href": "/usuarios/123"
14     },
15     "all_users": {
16       "href": "/usuarios"
17     },
18     "create_user": {
19       "href": "/usuarios"
20     }
21   }
22 }
```




```
1 GET /usuarios?page=1&page_size=5
```



```
1 {  
2   "usuarios": [ ... ],  
3   "_links": {  
4     "self": { "href": "/usuarios?page=1&page_size=5" },  
5     "next": { "href": "/usuarios?page=2&page_size=5" },  
6     "prev": null,  
7     "last": { "href": "/usuarios?page=3&page_size=5" }  
8   },  
9   "meta": { "total": 15, "page": 1, "page_size": 5, "total_pages": 3 }  
10 }  
11
```

HATEOAS

HATEOAS é uma abordagem poderosa para criar **APIs** flexíveis, escaláveis e fáceis de manter. Ele reduz o acoplamento entre cliente e servidor e melhora a navegação e descoberta de recursos.

Isso torna a interação com a **API** mais intuitiva e menos propensa a erros, além de proporcionar uma experiência mais robusta para o consumidor da **API**.

